Hotel Reservation Management System

A Relational Database Design and Implementation Project

Full Name: [INSERT YOUR FULL NAME]
Student ID: [INSERT YOUR STUDENT ID]
Module / Course: [INSERT MODULE NAME AND CODE]
Submission Date: [INSERT DATE]

GitHub Repository: https://github.com/INSERT-USERNAME/hotel-reservation-db
Video Demonstration: https://youtu.be/INSERT-VIDEO-ID

1. Introduction

Hotels generate a constant stream of transactional data: which guest is staying in which room, for how long, what extra services they used, and how they paid. Managing this with spreadsheets becomes unreliable once booking volume grows, because there is no enforced link between a guest, a room, and a payment, and duplicate or inconsistent records appear easily. This project addresses that problem by designing and implementing a relational database for a hotel reservation system, built so that the rules of the business (a room cannot be booked twice for overlapping needs without being tracked, a payment must belong to a real booking, a checkout date must come after a check-in date) are enforced by the database itself rather than left to whichever application happens to write to it.

The objectives of the project are to: (1) design a normalised relational schema that models customers, staff, rooms, bookings, services and payments without redundant or inconsistent data; (2) implement that schema in SQL with explicit primary keys, foreign keys and constraints; (3) populate it with realistic sample data; (4) demonstrate the system's functionality through a representative set of SQL operations covering insertion, update, deletion, joins, aggregation and subqueries; and (5) document the design decisions, the resulting structure and the issues encountered during development.

2. System Design

The schema is organised around nine tables. Customers, Staff, RoomTypes, Rooms and Services are reference entities that change relatively rarely. Bookings, BookingRooms, BookingServices and Payments are transactional entities that record what actually happens over time. The relationships are:

- A customer can make many bookings, but each booking belongs to exactly one customer (one-to-many).
- A staff member can process many bookings; a booking is processed by at most one staff member, and the link is optional (one-to-many, nullable foreign key).
- A room type (Standard, Deluxe, Suite) groups many physical rooms (one-to-many).
- A booking can include several rooms, and a room is reused across many bookings over its lifetime, so Bookings and Rooms are linked through the junction table BookingRooms (many-to-many).
- A booking can include several optional add-on services (breakfast, spa access, etc.), and a service can be attached to many bookings, linked through BookingServices (many-to-many).
- A booking can be settled by one or more payments, for example a deposit followed by a balance (one-to-many).

Figure 1 shows the resulting entity-relationship diagram, including the primary key (PK) and foreign key (FK) of every table.

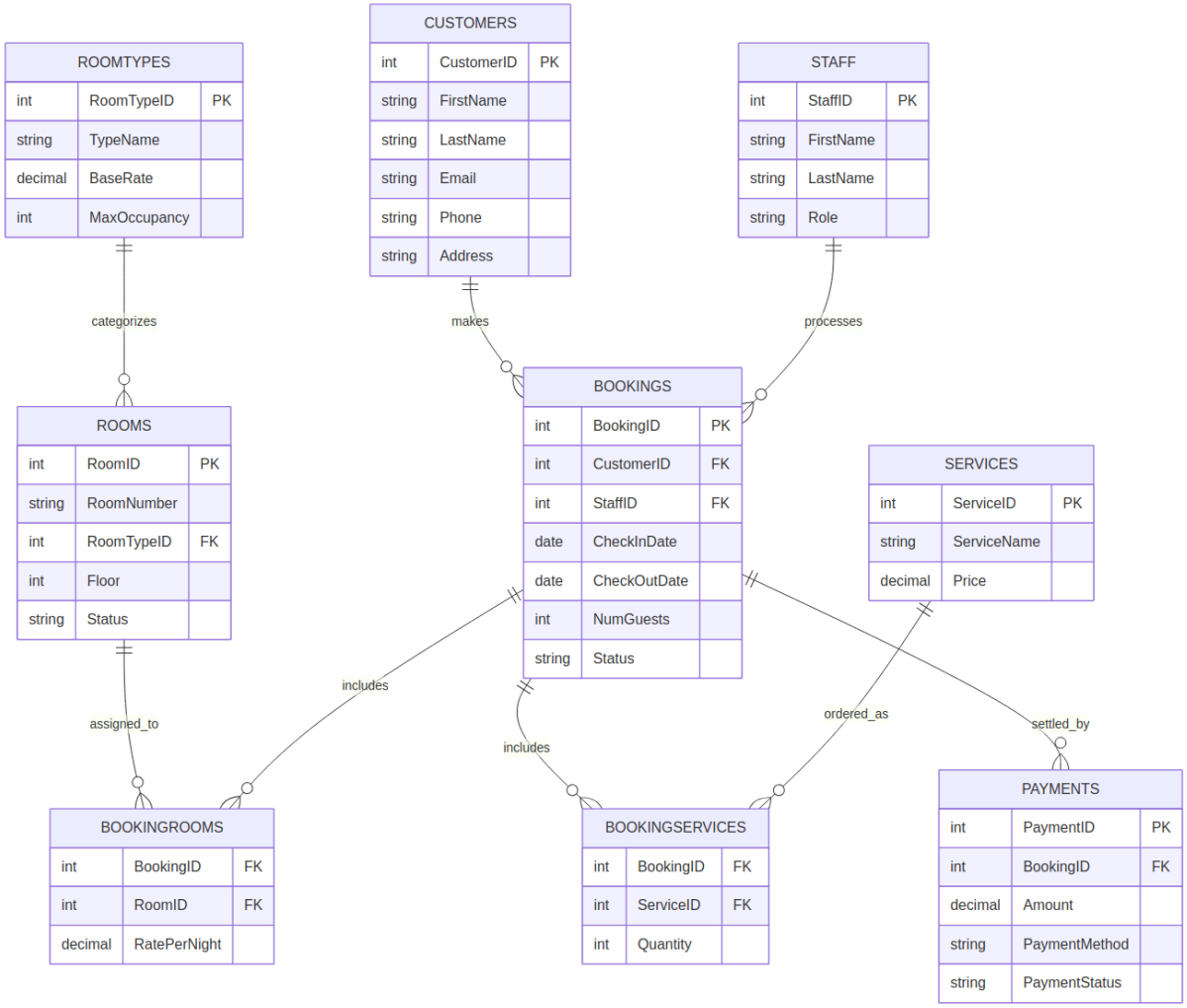


Figure 1: Entity-relationship diagram of the hotel reservation database

The design follows third normal form (3NF). First normal form is satisfied because every column holds a single atomic value; in particular, a booking does not store a comma-separated list of room numbers — that repeating group is removed by representing each booking-room pair as its own row in BookingRooms. Second normal form is satisfied because every junction table uses a composite primary key (for example BookingID + RoomID), and the only non-key attribute on that table, RatePerNight, depends on the combination of both keys, not on either one alone. Third normal form is satisfied because no non-key attribute depends on another non-key attribute: for instance, a room's nightly rate is stored once in RoomTypes and referenced by RoomTypeID, rather than being copied onto every row in Rooms or Bookings, which would risk the copies drifting out of sync.

3. Implementation

The database was implemented in MySQL 8.0 (InnoDB) using the script 01_schema.sql in the project repository. Tables were created in an order that respects foreign key dependencies: independent reference tables first (Customers, Staff, RoomTypes, Services), then tables that reference them (Rooms, Bookings), and finally the two junction tables and Payments, which depend on Bookings.

3.1 Keys and constraints

Every table has a single-column surrogate primary key (an AUTO_INCREMENT integer) except the two junction tables, BookingRooms and BookingServices, which use a composite primary key made up of the two foreign keys they connect. This is the standard way to model a many-to-many relationship and also prevents the same room or service being attached to the same booking twice, since the composite key must be unique.

Foreign keys are given different ON DELETE behaviour depending on what the relationship means. Rows in BookingRooms, BookingServices and Payments are deleted automatically (ON DELETE CASCADE) when their parent booking is deleted, because they have no meaning without that booking. Bookings.StaffID is set to NULL if a staff record is removed (ON DELETE SET NULL), since a booking should still exist even if we lose track of who processed it. Rooms.RoomTypeID and Bookings.CustomerID use ON DELETE RESTRICT, which blocks deletion of a room type or customer that is still referenced by existing rows, protecting historical records from being silently invalidated.

CHECK constraints enforce rules that are part of the business logic, not just the data type: BaseRate, RatePerNight and Amount must be positive; NumGuests and Quantity must be positive; Status columns are restricted to a fixed set of values (for example Bookings.Status can only be Confirmed, CheckedIn, CheckedOut or Cancelled); and a dedicated constraint on Bookings guarantees CheckOutDate is strictly after CheckInDate. Email and RoomNumber are marked UNIQUE so the same address or physical room cannot be entered twice. The example below shows the Bookings table, which combines most of these constraint types in one place:

```
CREATE TABLE Bookings (
    BookingID     INT AUTO_INCREMENT PRIMARY KEY,
    CustomerID    INT NOT NULL,
    StaffID       INT,
    CheckInDate   DATE NOT NULL,
    CheckOutDate  DATE NOT NULL,
    NumGuests     INT NOT NULL CHECK (NumGuests > 0),
    Status        VARCHAR(20) NOT NULL DEFAULT 'Confirmed'
                  CHECK (Status IN
('Confirmed','CheckedIn','CheckedOut','Cancelled')),
    CONSTRAINT fk_bookings_customer FOREIGN KEY (CustomerID)
        REFERENCES Customers(CustomerID) ON DELETE RESTRICT,
    CONSTRAINT fk_bookings_staff FOREIGN KEY (StaffID)
        REFERENCES Staff(StaffID) ON DELETE SET NULL,
    CONSTRAINT chk_dates CHECK (CheckOutDate > CheckInDate)
);
```

3.2 Sample data and supporting objects

The script 02_sample_data.sql populates the database with five customers, three staff members, three room types, seven rooms, four services, five bookings, and the corresponding BookingRooms, BookingServices and Payments rows, including one cancelled booking, so that every status value and relationship in the schema is exercised by at least one row. Two indexes were added beyond the automatic ones on primary and unique keys — on Bookings(CheckInDate, CheckOutDate) and Rooms(Status) — since availability checks by date range and by status are the most frequent lookups in a reservation system. A view, ActiveBookingsSummary, pre-joins customer, room and payment information for bookings that are still Confirmed or CheckedIn, so that a reporting query does not need to repeat the same five-table join every time.

4. Results

The script 03_queries.sql contains eighteen SQL statements organised into insertion, update, deletion, join, aggregation, subquery and view sections, demonstrating that the schema supports the full range of operations required by a reservation system. A representative subset is shown below; the complete set, with comments explaining each query's purpose, is in the repository.

4.1 Multi-table join: full booking detail

This query joins five tables (Bookings, Customers, BookingRooms, Rooms, RoomTypes) to reconstruct, for each booking, which customer made it and which room(s) and room type were involved. It confirms that the junction table correctly returns one row per room within a booking — for example, the booking with two suites produces two rows, one per room.

BookingID	Customer	Room	Room type	Status
1	Anna Schmidt	201	Deluxe	Confirmed
2	Liam O'Connor	101	Standard	Confirmed
3	Sofia Rossi	301	Suite	CheckedOut
3	Sofia Rossi	302	Suite	CheckedOut
4	Kenji Tanaka	202	Deluxe	Confirmed

4.2 Aggregation: total paid per customer

This query joins Customers, Bookings and Payments, filters to completed payments only, and groups by customer to compute total revenue collected. It demonstrates that SUM and GROUP BY correctly attribute payments to the right customer through two levels of join.

Customer	Total paid (EUR)
Anna Schmidt	440.00
Liam O'Connor	160.00
Sofia Rossi	880.00
Kenji Tanaka	650.00

4.3 Aggregation with HAVING: multi-room bookings

This query groups BookingRooms by BookingID and uses HAVING COUNT(*) > 1 to isolate bookings that cover more than one room — the case the BookingRooms junction table exists specifically to support.

BookingID	Rooms in booking
3	2

Each result set was checked against the sample data by hand to confirm the row counts and values were correct before being included here. (When you run the project for submission, replace the tables above with screenshots taken directly from your SQL client — e.g. MySQL Workbench or phpMyAdmin — showing the same queries executed against the live database, as your module's marking criteria expect actual evidence of execution.)

5. Challenges and Solutions

Modelling multi-room bookings: an early version of the schema gave Bookings a single RoomID, which could not represent a guest booking several rooms in one transaction (as happened with Sofia Rossi's two-suite booking). This was resolved by introducing the BookingRooms junction table with a composite primary key, which also has the side benefit of recording the rate that applied to each room at the time of booking, independent of later changes to RoomTypes.BaseRate.

Choosing delete behaviour for foreign keys: applying ON DELETE CASCADE everywhere would let a careless deletion of a customer silently wipe out their entire booking and payment history, while applying ON DELETE RESTRICT everywhere would make it impossible to delete a cancelled booking's now-irrelevant child rows. The solution was to differentiate by relationship: CASCADE only for rows that are meaningless without their parent booking (BookingRooms, BookingServices, Payments), RESTRICT for reference data that historical records still point to (RoomTypes, Customers, Rooms), and SET NULL for an optional link (Bookings.StaffID).

Enforcing date logic at the database level: it would have been simpler to validate that CheckOutDate is after CheckInDate only in application code, but that protection disappears the moment a second application, script or manual SQL statement writes to the database. Adding a CHECK constraint on the table itself means the rule holds regardless of which client is used.

Validating logic before running everything on the target server: rather than writing all eighteen queries against MySQL and only then discovering a typo or a logic error, the schema, sample data and queries were first executed against SQLite, with MySQL-only syntax (AUTO_INCREMENT, ENGINE=InnoDB) substituted for its closest SQLite equivalent purely for testing purposes. This caught referential and aggregation issues quickly, before the final run against the real MySQL instance used for the project deliverable.

6. Conclusion and Future Work

This project produced a normalised, nine-table relational database for a hotel reservation system, with primary keys, foreign keys, unique constraints and check constraints enforcing the core business rules directly in the schema. The accompanying SQL script demonstrates that the design supports the full range of required operations — insertion, update, deletion, multi-table joins, aggregation with GROUP BY and HAVING, correlated and uncorrelated subqueries, and a view for simplified reporting — using realistic sample data covering every status value in the system.

Future work could extend the system in several directions: a Shifts or Rosters table to schedule staff rather than only recording who processed a booking; splitting Payments into planned instalments versus recorded transactions to support deposits and partial refunds more explicitly; a trigger or scheduled event to automatically update Rooms.Status to Occupied or Available as a booking's check-in and check-out dates are reached, instead of relying on a manual UPDATE; and a thin application or API layer on top of the schema so that front-desk staff could interact with the system through a form-based interface rather than writing SQL directly.